

ECE 456 Problem Set 2:

The Schrödinger Equation

Omar Mahmoud, Nasif Qadri

March 2, 2026

1 Numerical Solution for the Particle in a Box

1.1 Numerical Implementation and Eigenvalue Extraction

Consider the problem of a particle in an escape-proof box. The code in `1a.py` was used to find the eigenvectors and eigenvalues of the corresponding matrix equation $[\hat{\mathcal{H}}]\{\phi\} = E\{\phi\}$. The lattice points are defined from $n = 1$ to $n = N$, with $N = 100$ and lattice constant $a = 1 \text{ \AA}$.

When ran, `1a.py` generated the plots found in Figures 1 and 2.

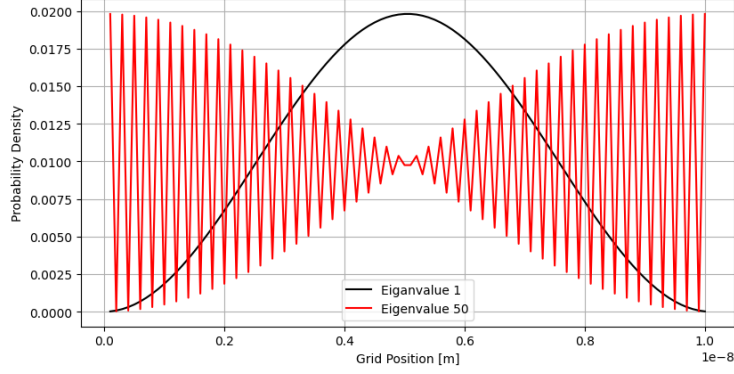


Figure 1: Position-probability densities from the numerical wave functions $\{\phi\}$ for eigenvalue numbers 1 and 50.

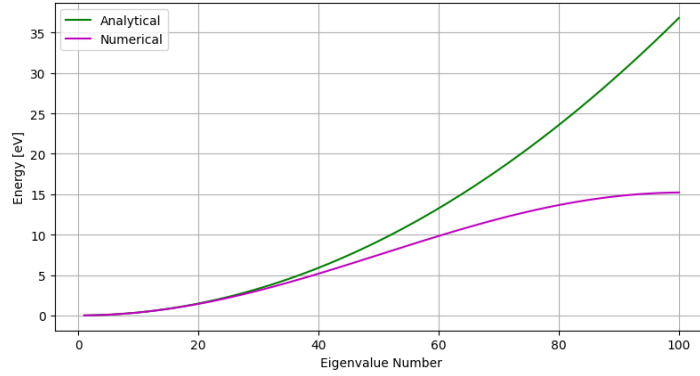


Figure 2: The 100 eigenvalues that resulted from the numerical and analytical solutions.

1.2 Normalization of Analytical and Numerical Representations

1.2.1 Normalization of the Analytical Wave Function

The analytical solution can be written as

$$\phi(x) = A \sin\left(\frac{n\pi}{L}x\right) \quad (1)$$

where A is a constant, and where the box extends from $x = 0$ to $x = L$. Normalization requires

$$\int_0^L |\phi(x)|^2 dx = 1 \quad (2)$$

The value of A required to properly normalize the analytical wave function can be found by

$$\begin{aligned}
 \int_0^L |\phi(x)|^2 dx &= 1 \\
 \int_0^L \left| A \sin\left(\frac{n\pi x}{L}\right) \right|^2 dx &= 1 \\
 \int_0^L A^2 \sin^2\left(\frac{n\pi x}{L}\right) dx &= 1 \\
 A^2 \left[\frac{1}{2}x - \frac{L}{4n\pi} \sin\left(\frac{2n\pi x}{L}\right) \right]_0^L &= 1 \\
 A^2 \left[\frac{x}{2} - \frac{L}{4n\pi} \sin\left(\frac{2n\pi x}{L}\right) \right]_0^L &= 1 \\
 A^2 \left[\left(\frac{L}{2} - \frac{L}{4n\pi} \sin(2n\pi) \right) - \left(0 - \frac{L}{4n\pi} \sin(0) \right) \right] &= 1 \\
 A^2 \left(\frac{L}{2} \right) &= 1 \\
 A^2 &= \frac{2}{L} \\
 \boxed{A = \sqrt{\frac{2}{L}}}
 \end{aligned}$$

1.2.2 Numerical Normalization and the Continuum Limit

For sufficiently small values of the lattice constant a , it is reasonable to assume that the ℓ th component of the numerical eigenvector $\{\phi\}$ will follow a functional form that is consistent with (2), and will be given, for example, by

$$\phi_\ell = B \sin\left(\frac{n\pi}{L} x_\ell\right) \quad (3)$$

where B is a normalization constant and $x_\ell = a\ell$ is the ℓ th lattice point. If the numerical eigenvectors are normalized according to the equation

$$\sum_{\ell=1}^N |\phi_\ell|^2 = 1 \quad (4)$$

the expected value of B in the limit that $a \rightarrow 0$ can be found by

$$\begin{aligned}
 \sum_{\ell=1}^N |\phi_\ell|^2 &= 1 \\
 a \sum_{\ell=1}^N |\phi_\ell|^2 &= a \\
 a \sum_{\ell=1}^N \left| B \sin\left(\frac{n\pi}{L} x_\ell\right) \right|^2 &= a
 \end{aligned}$$

As $a \rightarrow 0$ the total number of lattices $N \rightarrow \infty$ (Riemann sum), therefore this becomes:

$$\begin{aligned}
 \int_0^L B^2 \sin^2\left(\frac{n\pi x}{L}\right) dx &= a \\
 B^2 \left(\frac{L}{2} \right) &= a \\
 B^2 &= \frac{2a}{L} \\
 \boxed{B = \sqrt{\frac{2a}{L}}}
 \end{aligned}$$

1.3 Investigation of Numerical Eigenvalue Deviation

1.3.1 Derivation of the Discrete Dispersion Relation

The discretized time-independent Schrödinger equation is given by:

$$-t_0\phi_{\ell-1} + (2t_0 + U_\ell)\phi_\ell - t_0\phi_{\ell+1} = E\phi_\ell$$

For a particle in a box, the potential inside is zero ($U_\ell = 0$). We assume the ansatz $\phi_\ell = B \sin(kx_\ell) = B \sin(kal)$ with $k = \frac{n\pi}{L}$. Rearranging the difference equation:

$$-t_0(\phi_{\ell-1} + \phi_{\ell+1}) + 2t_0\phi_\ell = E\phi_\ell$$

Using the trigonometric addition and subtraction identities:

$$\begin{aligned}\phi_{\ell-1} &= B \sin(ka(\ell-1)) = B [\sin(ka\ell) \cos(ka) - \cos(ka\ell) \sin(ka)] \\ \phi_{\ell+1} &= B \sin(ka(\ell+1)) = B [\sin(ka\ell) \cos(ka) + \cos(ka\ell) \sin(ka)]\end{aligned}$$

Summing these terms cancels the cosine-sine terms:

$$\begin{aligned}\phi_{\ell-1} + \phi_{\ell+1} &= 2B \sin(ka\ell) \cos(ka) \\ &= 2\phi_\ell \cos(ka)\end{aligned}$$

Substituting this back into the rearranged difference equation:

$$\begin{aligned}-t_0(2\phi_\ell \cos(ka)) + 2t_0\phi_\ell &= E\phi_\ell \\ 2t_0\phi_\ell - 2t_0\phi_\ell \cos(ka) &= E\phi_\ell \\ 2t_0\phi_\ell(1 - \cos(ka)) &= E\phi_\ell\end{aligned}$$

Dividing both sides by ϕ_ℓ and substituting $k = \frac{n\pi}{L}$:

$$E = 2t_0 \left[1 - \cos\left(\frac{n\pi a}{L}\right) \right] \quad (5)$$

1.3.2 Validation of Results

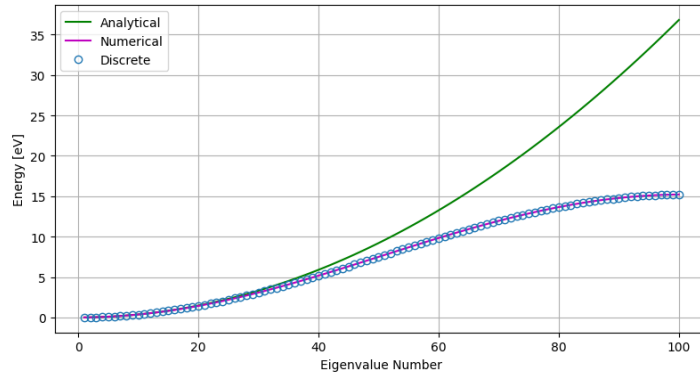


Figure 3: Comparison of the analytical energy eigenvalues (solid line), the numerical eigenvalues obtained via the finite difference method (x's), and the expected discrete eigenvalues derived in Equation 5 (circles).

The plot demonstrates that the discrete energy expression derived in Equation 5 matches the numerical eigenvalues (represented by the black x's) perfectly.

This confirms that the deviation of the numerical results from the analytical prediction ($E \propto n^2$) at higher energy levels ($n \geq 50$) is a predictable consequence of the discrete lattice. The finite difference method only approximates the continuous relation when n is small (when the wavelength is large compared to the lattice spacing a).

1.3.3 Small-Angle Approximation

For small angles θ , the cosine function can be approximated by the Taylor series expansion:

$$\cos \theta \approx 1 - \frac{\theta^2}{2}$$

Applying this to Equation 5 where $\theta = \frac{n\pi a}{L}$:

$$\begin{aligned} E &= 2t_0 \left[1 - \cos\left(\frac{n\pi a}{L}\right) \right] \\ &\approx 2t_0 \left[1 - \left(1 - \frac{1}{2} \left(\frac{n\pi a}{L} \right)^2 \right) \right] \\ &= 2t_0 \left[\frac{1}{2} \frac{n^2 \pi^2 a^2}{L^2} \right] \\ &= t_0 \frac{n^2 \pi^2 a^2}{L^2} \end{aligned}$$

Substituting $t_0 = \frac{\hbar^2}{2ma^2}$:

$$\begin{aligned} E &\approx \left(\frac{\hbar^2}{2ma^2} \right) \frac{n^2 \pi^2 a^2}{L^2} \\ &= \frac{\hbar^2 \pi^2 n^2}{2mL^2} \end{aligned}$$

The result is consistent with the plot in Figure 3. For small values of n (where the angle $\frac{n\pi a}{L}$ is small), the discrete energy expression reduces exactly to the analytical eigenvalue expression $E = \frac{\hbar^2 \pi^2 n^2}{2mL^2}$. This explains why the numerical and analytical results overlap at low energy levels.

1.3.4 Grid Refinement and Resolution of Sampling Errors

By reducing the lattice spacing a by a factor of 10 and increasing N to 1000, the numerical eigenvalues now match the analytical results almost perfectly up to $n = 100$. The deviation seen in Figure 2 for $n \geq 50$ has vanished. This is because the finer grid provides a better sampling rate, keeping the argument of the cosine in the discrete expression, $\frac{n\pi a}{L}$, small enough that the approximation $\cos(x) \approx 1 - x^2/2$ remains valid for a larger range of n .

1.4 Periodic Boundary Conditions

1.4.1 TITLE HERE

The code in `1d.py` is a modified version of `1a.py` changed to implement periodic boundary conditions. Running `1d.py` to get the eigenvalue numbers 4 and 5 produced the output:

```
Eigenvalue 4: 0.060023108288934125 eV
Eigenvalue 5: 0.06002310828893526 eV
```

The energy for eigenvalue numbers 4 and 5 is degenerate because both eigenvalues correspond to nearly identical energies. This shows that each energy level (above the ground state) has two distinct wave-functions (states) associated with it.

The eigenvalue numbers 1 through 5 can be produced by the same code:

```
Eigenvalue 1: -1.431884637890359e-15 eV
Eigenvalue 2: 0.0150205969306892 eV
Eigenvalue 3: 0.015020596930693126 eV
Eigenvalue 4: 0.060023108288934125 eV
Eigenvalue 5: 0.06002310828893526 eV
```

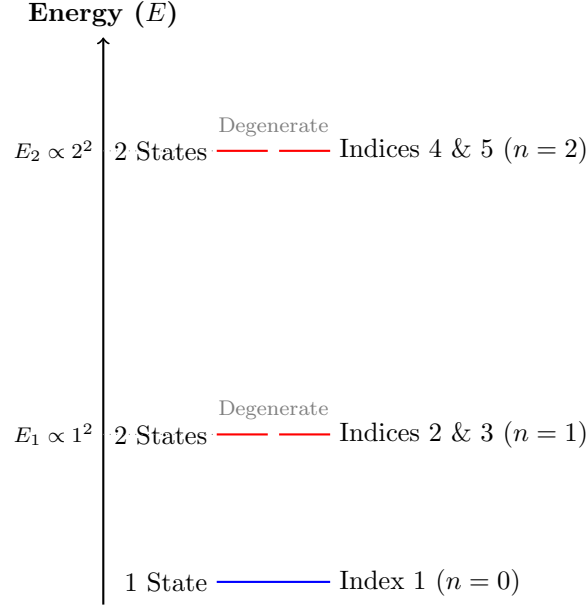


Figure 4: Energy-level diagram for a particle in a ring (periodic boundary conditions). Note the single ground state (Index 1) followed by pairs of degenerate states (Indices 2-3, 4-5).

1.4.2 Comparison with Analytical Results

The analytical eigenvalues for a particle in a ring are given by the equation:

$$E_{\text{analytical}} = \frac{2\hbar^2 \pi^2 n^2}{mL^2} \quad (6)$$

where n is analytical eigenvalue number. The mapping between the numerical eigenvalue index and the analytical quantum number n is as follows:

- Index 1 corresponds to $n = 0$.
- Indices 2 and 3 correspond to $n = 1$.
- Indices 4 and 5 correspond to $n = 2$.

The comparison between the analytical values and the numerical results is summarized in Table 1.

Table 1: Comparison of numerical and analytical eigenvalues for the particle in a ring

Index	Numerical E [eV]	Analytical E [eV]	Ring n	Degeneracy
1	0.0000	0.0000	0	Non-Degenerate
2	0.0148	0.0149	1	Degenerate
3	0.0148	0.0149	1	
4	0.0591	0.0596	2	Degenerate
5	0.0591	0.0596	2	

The energies for indices 2 and 3 are virtually identical, as are the energies for indices 4 and 5, confirming the degeneracy expected for a particle in a ring. Small discrepancies between numerical and analytical values can be chalked up to the finite lattice spacing a . Therefore, the numerical results are consistent with the analytical predictions.

2 Radial Equation for the Hydrogen Atom

The radial part of the solution of the Schrödinger equation for the hydrogen atom can be written in terms of a function $f(r)$ where $|f(r)|^2 dr$ represents the probability of finding the electron in the volume

between r and $r + dr$, where the nucleus is at the origin $r = 0$. Given that $f(r)$ is governed by the radial Schrödinger equation,

$$\left[\frac{-\hbar^2}{2m} \frac{d^2}{dr^2} - \frac{q^2}{4\pi\epsilon_0 r} + \frac{l_o(l_o + 1)\hbar^2}{2mr^2} \right] f(r) = Ef(r) \quad (7)$$

where $\epsilon_0 = 8.854 \times 10^{-12} \text{ F m}^{-1}$.

2.1 Defining the Hamiltonian Matrix

If Equation 7 is rewritten in the form

$$\hat{\mathcal{H}}f(r) = Ef(r) \quad (8)$$

with

$$\hat{\mathcal{H}} \equiv \left[\frac{-\hbar^2}{2m} \frac{d^2}{dr^2} - \frac{q^2}{4\pi\epsilon_0 r} + \frac{l_o(l_o + 1)\hbar^2}{2mr^2} \right] \quad (9)$$

and boundary conditions are such that $f(r) = 0$ at the endpoints of the discretized space, $[\hat{\mathcal{H}}]$ would be defined as

$$[\hat{H}] = \begin{bmatrix} 2t_0 + U_1 & -t_0 & 0 & \dots & 0 \\ -t_0 & 2t_0 + U_2 & -t_0 & \dots & 0 \\ 0 & -t_0 & 2t_0 + U_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 2t_0 + U_N \end{bmatrix} \quad (10)$$

where U_l and t_0 are defined as

$$U_l = -\frac{q^2}{4\pi\epsilon_0 r_n} + \frac{l_o(l_o + 1)\hbar^2}{2mr_n^2} \quad (11)$$

$$t_0 = \frac{\hbar^2}{2m(\Delta r)^2} \quad (12)$$

The code in `1a.py` was modified to what is seen in `2c.py`. `2c.py` was used to produce the plots for the first two eigenvectors, corresponding to the 1s and 2s energy states, as seen in Figures [ADD REF HERE].

Eigenvalue 1: -13.497781923462465 eV

Eigenvalue 2: -3.396072886508772 eV

3 Appendix

```

1  #!/usr/bin/env python3
2
3  import matplotlib.pyplot as plt
4  import numpy as np
5
6  # Physical constants in MKS units
7  hbar = 1.054e-34 # Reduced Planck constant [J*s]
8  q = 1.602e-19 # Elementary charge [C]
9  m = 9.110e-31 # Free-electron mass [kg]
10
11 # Lattice data
12 N = 100 # Total number of internal lattice points [unitless integer]
13 n = np.linspace(1, N, N) # Array of lattice point indices/eigenvalue numbers [unitless]
14 a = 1e-10 # Lattice constant or distance between grid points [m]
15 x = a * np.linspace(1, N, N) # Discretized position grid [m]
16 t0 = (
17     (hbar**2) / (2 * m * a**2) / q
18 ) # Hopping parameter / coupling energy, divided by q to convert to [eV]
19 L = a * (N + 1) # Total length of the 1D lattice (infinite square well) [m]
20 lattice_points = np.linspace(0, N + 1) * a
21 U = np.zeros(len(x))
22 diag1 = np.diag(np.array([-t0 for i in range(N - 1)]), -1)
23 diag2 = np.diag(np.array([2 * t0 for i in range(N)] + U, 0)
24 diag3 = np.diag(np.array([-t0 for i in range(N - 1)]), 1)
25 H = diag1 + diag2 + diag3
26
27 evals, evects = np.linalg.eigh(H)
28
29 pd1 = evects[:, 1 - 1] ** 2
30 pd50 = evects[:, 50 - 1] ** 2
31 plt.figure()
32 plt.plot(x, pd1, "k-", label="Eigenvalue 1")
33 plt.plot(x, pd50, "r-", label="Eigenvalue 50")
34 plt.xlabel("Grid Position [m]")
35 plt.ylabel("Probability Density")
36 plt.legend()
37 plt.grid()
38 plt.show()
39
40 E = (hbar * np.pi * n) ** 2 / (2 * m * L**2) / q
41 plt.figure()
42 plt.plot(n, E, "g-", label="Analytical")
43 plt.plot(n, evals, "m-", label="Numerical")
44 plt.plot(
45     n,
46     2 * t0 * (1 - np.cos(n * np.pi * a / L)),
47     "o",
48     markerfacecolor="none",
49     label="Discrete",
50 )
51 plt.xlabel("Eigenvalue Number")
52 plt.ylabel("Energy [eV]")
53 plt.legend()
54 plt.grid()
55 plt.show()
56
57 with np.printoptions(precision=3, suppress=True):
58     print(np.linalg.eigh(H))

```

Listing 1: Code in 1a.py used to generate Figures 1 and 2.


```

1  #!/usr/bin/env python3
2
3  import matplotlib.pyplot as plt
4  import numpy as np
5
6  # Physical constants in MKS units
7  hbar = 1.054e-34 # Reduced Planck constant [J*s]
8  q = 1.602e-19 # Elementary charge [C]
9  m = 9.110e-31 # Free-electron mass [kg]
10
11 # Lattice data
12 N = 100 # Total number of internal lattice points [unitless integer]
13 n = np.linspace(1, N, N) # Array of lattice point indices/eigenvalue numbers [unitless]
14 a = 1e-10 # Lattice constant or distance between grid points [m]
15 x = a * np.linspace(1, N, N) # Discretized position grid [m]
16 t0 = (
17     (hbar**2) / (2 * m * a**2) / q
18 ) # Hopping parameter / coupling energy, divided by q to convert to [eV]
19 L = a * (N + 1) # Total length of the 1D lattice (infinite square well) [m]
20 lattice_points = np.linspace(0, N + 1) * a
21 U = np.zeros(len(x))
22 diag1 = np.diag(np.array([-t0 for i in range(N - 1)]), -1)
23 diag2 = np.diag(np.array([2 * t0 for i in range(N)]) + U, 0)
24 diag3 = np.diag(np.array([-t0 for i in range(N - 1)]), 1)
25 H = diag1 + diag2 + diag3
26
27 # Periodic boundary conditions
28 H[0, -1] = -t0
29 H[-1, 0] = -t0
30
31 evals, evects = np.linalg.eigh(H)
32 pd1 = evects[:, 1 - 1] ** 2
33 pd50 = evects[:, 50 - 1] ** 2
34 pd4 = evects[:, 4 - 1] ** 2
35 pd5 = evects[:, 5 - 1] ** 2
36 print(f"Eigenvalue 1: {evals[1 - 1]} eV")
37 print(f"Eigenvalue 2: {evals[2 - 1]} eV")
38 print(f"Eigenvalue 3: {evals[3 - 1]} eV")
39 print(f"Eigenvalue 4: {evals[4 - 1]} eV")
40 print(f"Eigenvalue 5: {evals[5 - 1]} eV")
41
42 plt.figure()
43 plt.plot(x, pd4, "k-", label="Eigenvalue 4")
44 plt.plot(x, pd5, "r-", label="Eigenvalue 5")
45 plt.xlabel("Grid Position [m]")
46 plt.ylabel("Probability Density")
47 plt.legend()
48 plt.grid()
49 plt.show()
50
51 plt.figure()
52 plt.plot(n, evals, "m-", label="Numerical")
53 plt.xlabel("Eigenvalue Number")
54 plt.ylabel("Energy [eV]")
55 plt.legend()
56 plt.grid()
57 plt.show()

```

Listing 2: Code in 1d.py used to generate Figures.